HASHEVICT: A PRE-ATTENTION KV CACHE EVICTION STRATEGY USING LOCALITY-SENSITIVE HASHING

A PREPRINT

Minghui Liu *
University of Maryland
minghui@umd.edu

Tahseen Rabbani *
Yale University
tahseen.rabbani@yale.edu

Tony O'Halloran
University of Galway
t.ohalloran8@universityofgalway.ie

Ananth Sankaralingam
University of Maryland
ananths1@terpmail.umd.edu

Mary-Anne Hartley
Yale University
mary-anne.hartley@yale.edu

Furong Huang
University of Maryland
Capital One
furongh@cs.umd.edu

Cornelia Fermüller
University of Maryland
fermulcm@umd.edu

Yiannis Aloimonos
University of Maryland
jyaloimo@umd.edu

ABSTRACT

The key-value (KV) cache in large language models (LLMs) markedly accelerates attention and inference. However, since it requires storing the key and value embeddings of past tokens it can quickly consume a significant amount of GPU memory. In this work, we introduce HASHEVICT, a KV cache compression strategy that uses locality-sensitive hashing (LSH) to efficiently estimate attention and evict unimportant tokens. At every decoding step, the key and value of the current token replace the embeddings of a token estimated to produce the lowest attention score. We project key and query embeddings to dimensionally-reduced binary hash codes and locate low-attention tokens via a lightweight Hamming distance calculation, all of which is efficiently computable on the GPU. Unlike existing compression strategies that compute attention to determine token retention, HASHEVICT makes these decisions pre-attention, thereby reducing computational costs. We demonstrate that HASHEVICT can compress the KV cache by 30%-70% while maintaining high performance across reasoning, multiple-choice, long-context retrieval and summarization tasks. Our method is fast: we achieve 1.5-2x prefill speed and competitive decoding speed against baseline methods such as H_2O, Scissorhands, and 17$\times$ prefill/2x decoding speed against FastGen.

Keywords KV cache · locality-sensitive hashing · efficient LLM inference · token eviction

1 Introduction

The advent of large language models (LLMs) has enabled sharp improvements over innumerable downstream natural language processing (NLP) tasks, such as summarization and dialogue generation [Zhao et al., 2023, Wei et al., 2022]. The hallmark feature of LLMs, the attention module [Bahdanau, 2014, Luong, 2015, Vaswani, 2017], enables contextual processing over sequences of tokens. To avoid repeated dot products over key and value embeddings of tokens, a key-value (KV) cache is maintained in VRAM to maintain these calculations. This technique is particularly popular with decoder LLMs.

However, the size of the KV cache scales quadratically with sequence length n and linearly with the number of attention layers and heads. Assuming the size of the KV cache is n tokens, for each new decoded token, n attention scores

*Equal contributions.

need to be added which requires a total of $\mathcal{O}(dn^2)$ computation, where d is the projection dimension, and $\mathcal{O}(n^2)$ storage. For example, maintaining the KV cache for a sequence of 4K tokens in half-precision (FP16) can require approximately ∼16GB of memory for most models within the Llama 3 family [Dubey et al., 2024]. These memory costs are exacerbated with batched inference and result in high decoding latency [Fu, 2024]. Consequently, there is significant interest in compressing the size of the KV cache to enable longer context windows and low-resource, on-device deployment.

An emerging strategy for reducing the size of the KV cache is *token eviction*. This approach drops the key and value embeddings for past tokens in the cache, skipping future attention calculations involving these tokens.

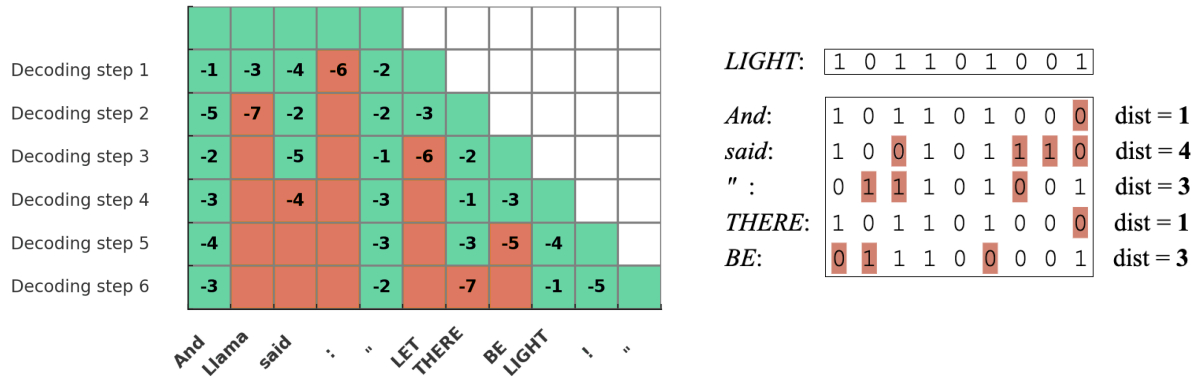
Various token eviction/retention policies have been explored in recent literature, including

the profiling of token type preferences [Ge et al., 2023], retention of heavy-hitter tokens [Zhang et al., 2024a,b], and dropping tokens based on the high L_2 norms of their key embeddings [Devoto et al., 2024].

The latter approach [Devoto et al., 2024] is intriguing as eviction decisions are performed pre-attention. However, this L_2 dropout strategy in inclined towards long-context retrieval tasks. It developed based on an empirical observation that smaller norm of key embedding correlates with higher attention score. For long-context retrieval tasks, high-attention score tokens are the most important tokens since the question's text will overlap with the piece of context that needs to be retrieved. Thus, it is specialized to retain only those tokens with the highest attention, which we find unsuitable for free response reasoning tasks. Existing literature suggests that retaining tokens with a diverse spectrum of attention scores (skewing high) is necessary [Guo et al., 2024, Zhang et al., 2024a, Long et al., 2023].

Is there a non-attentive KV cache compression strategy that is performant over a wide variety of tasks, including multiple-choice, summarization, long-context retrieval, and free response question-answering? This work answers this question positively by introducing a novel strategy, HASHEVICT, that *dynamically* determines token eviction pre-attention via locality-sensitive hashing (LSH) [Goemans and Williamson, 1995, Charikar, 2002]. HASHEVICT evicts a past token from the cache whose key embedding is highly cosine dissimilar to the current query token embedding. The intuition behind this strategy is that high cosine dissimilarity indicates a low dot-product attention score. To efficiently scan for cosine (dis)similar tokens without performing attention, HASHEVICT leverages the SimHash [Charikar, 2002, Goemans and Williamson, 1995] to instead compare Hamming distances between c-length binary hashes of cached key embeddings and the current query embedding. We depict a high-level visualization of this strategy in Figure 1.

HASHEVICT requires minimal overhead: for a total sequence length of ℓ tokens with embedding dimension d, HASHEVICT maintains a constant-size, low-cost binary array in GPU memory of size $c \times k$ bytes, where $c \ll d$ is the hash dimension and $k \ll \ell$. Cached tokens with key embeddings that register low Hamming similarity measurements to decoded query embeddings are gradually replaced.



(a) KV cache during decoding

(b) LSH comparison at decoding step 4

Figure 1: **An abstract visualization of HASHEVICT eviction strategy.** Figure 1a depicts the strategy for several decoding steps. The cache can only maintain 5 tokens due to memory constraints. At each decoding step, HASHEVICT projects the query embedding of the current token i and all previous key embeddings to *binary hash codes*. HASHEVICT then measures the negative of Hamming distances between the query *code* of token i and key *codes* of all tokens j in the cache. Each step, HASHEVICT evicts the key/values of the token with the lowest score (marked as red) from the cache. Figure 1b depicts the LSH comparison for decoding step 4, marking the token *"said"* for removal, as its high Hamming indicates low cosine similarity (and thus, low attention).

Our contributions are as follows:

- **Novel Attention-Free Token Eviction** We introduce a novel *attention-free* token eviction strategy, HASHEVICT, that leverages locality-sensitive hashing (LSH) to quickly locate which token in the cache is the least relevant to the current query. This ranking procedure consists entirely of cheap Hamming distance calculations. The associated binary array for computing these similarities requires minimal memory overhead. For a Llama 3 model, HASHEVICT can compress the KV cache by 30%-70% with minimal performance drop
- **State-of-the-Art Performance** HASHEVICT demonstrates high performance on reasoning tasks (GSM8K Cobbe et al. [2021], MedQA Cobbe et al. [2021]), multiple-choice (GSM8K MC, MedQA MC), long-context retrieval (Needle-in-a-Haystack, Common Word [Hsieh et al., 2024]), and long-text summarization (MultiNews, GovReport Bai et al. [2023]). To the best of our knowledge, HASHEVICT achieves state-of-the-art performance for attention-free eviction, outperforming the similar attention-free L_2 method. Additionally, HASHEVICT outperforms attention-accumulation-based methods on long text summarization tasks and achieves 1.5x speedup in the prefilling stage and comparable speed in the decoding stage withoug low-level optimiations.
- **Open-Source Implementation** Upon public release of our manuscript, we will release an open-source implementation of HASHEVICT through a fork of the popular cold-compress library (`https://github.com/AnswerDotAI/cold-compress`).

2 Preliminaries

We aim to capture tokens whose query embeddings will form a large sum of dot products (i.e., attention scores) with other key embeddings, but without explicitly calculating attention. We will leverage locality-sensitive hashing (LSH) to quickly determine cosine similarities since the angle is equivalent to the dot product (for unit vectors). In this section, we review technical concepts crucial to attention and locality-sensitive hashing. We assume some base level of similarity with transformers, but we refer the reader to precise formalism [Phuong and Hutter, 2022].

Scaled Dot-Product Attention Consider a sequence of n tokens with e-dimensional real-valued representations $x_1, x_2, \ldots, x_n$. Let $Q = [q_1\, q_2\, \cdots\, q_n] \in \mathbb{R}^{n\times d}$, $K = [k_1 k_2 \cdots k_n] \in \mathbb{R}^{d\times n}$ where $q_i = W_q x_i$, $k_i = W_k x_i$ and $W, K \in \mathbb{R}^{d\times e}$. The query and key projectors W_q and W_k are pre-trained weight matrices. We also define a value matrix $V = [v_1 v_2 v_2 \cdots v_n] \in \mathbb{R}^{d_{out}\times n}$ with $v_i = W_v x_i$ with trainable $V \in \mathbb{R}^{d_{out}\times d}$, the scaled dot-product attention mechanism is given as

$$\text{Attention}(Q, K, V) = V \cdot \text{softmax}\Big(\frac{Q^\top K}{\sqrt{d}}\Big). \tag{1}$$

Typically, attention layers contain multiple heads $\{h_i\}_{i=1}^J$ each with distinct query, key, and value projectors $\{W_q^{(h_i)}, W_k^{(h_i)}, W_v^{(h_i)}\}_{i=1}^J$. In a multi-head setup, attention is computed in parallel across all heads, and the outputs are concatenated together and then passed through a linear layer for processing by the next transformer block.

As Q, K, V are updated with each new incoming token, to avoid significant re-computation, the current state of $Q^\top K$, Q, and K are maintained in the KV cache. Our goal is to bypass attention computation and caching for select tokens, i.e., sparsify the attention matrix $Q^\top K$, K, and V.

Locality-Sensitive Hashing We will now describe a family of locality-sensitive hashing (LSH) functions able to efficiently approximate nearest neighbors (per cosine similarity) of key/query vectors in high-dimensional $\mathbb{R}^d$ through comparison in a reduced c-dimensional space (per Hamming distance) with $c \ll d$. Here, "locality-sensitive" means points that are close together according to a distance function $\text{dist}_d(\cdot,\cdot)$ in the ambient space remain close per another distance function $\text{dist}_c(\cdot,\cdot)$ in the lower-dimensional space with high-probability. For a rigorous treatment of LSH functions, see [Andoni et al., 2018, Charikar, 2002].

Formally for our setup, $dist_d(x, y) \triangleq \cos\theta_{x,y} = \frac{x^\top y}{||x||\,||y||}$ and $dist_c(p, q) \triangleq \mathrm{d}_H(p, q)$ which denotes the Hamming distance. We will project each vector from $\mathbb{R}^d$ into $\mathbb{Z}_2^c$, the space of c-bit binary strings (which is often referred to as a *binary hash code*). To acquire a c-bit long hash code from an input vector $x \in \mathbb{R}^d$, we define a random projection matrix $R \in \mathbb{R}^{c\times d}$ whose entries are independently sampled from the standard normal distribution $\mathcal{N}(0, 1)$. We then define

$$h(x) = \text{sgn}(Rx), \tag{2}$$

where $\text{sgn}(\cdot)$ (as an abuse of conventional notation) is the element-wise Heaviside step function:

$$\text{sgn}(x) := \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}.$$